

Scenariusz wypowiedzi do prezentacji pracy inżynierskiej

Zaawansowana aplikacja webowa do organizacji zadań

Piotr Słupski

Promotor: dr Leszek Grocholski

Spis treści

Slajd 1. Strona tytułowa	4
Slajd 2. Spis treści	4
Slajd 3. Cel pracy	4
Slajd 4. Zakres pracy	5
Slajd 5. Wymagania funkcjonalne	5
Slajd 6. Uzupełnienie wymagań funkcjonalnych	6
Slajd 7. Wymagania нефunkcjonalne	6
Slajd 8. Podobne aplikacje	6
Slajd 9. Uzupełnienie podobnych aplikacji	7
Slajd 10. Podstawy technologiczne frontendu	7
Slajd 11. Podstawy technologiczne backendu	8
Slajd 12. Autentykacja i autoryzacja	8
Slajd 13. Wybór bazy danych	8
Slajd 14. Ogólna architektura aplikacji	9
Slajd 15. Diagram komponentów aplikacji	9
Slajd 16. Diagramy przepływu danych	10

Slajd 17. Diagram przepływu danych poziomu 1	10
Slajd 18. Schemat bazy danych	10
Slajd 19. Najważniejsze mechanizmy aplikacji	11
Slajd 20. Testy	11
Slajd 21. Demonstracja aplikacji	11
Slajd 22. Podsumowanie	12
Slajd 23. Bibliografia	12
Slajd 24. Dziękuję za uwagę	12

Slajd 1. Strona tytułowa

Dzień dobry, nazywam się Piotr Słupski i chciałbym przedstawić pracę inżynierską pod tytułem „Zaawansowana aplikacja webowa do organizacji zadań”. Praca została przygotowana pod kierunkiem dr. Leszka Grocholskiego.

Tematem projektu było stworzenie aplikacji internetowej, która pomaga użytkownikowi indywidualnemu porządkować zadania, checklisty, notatki oraz projekty. W prezentacji omówię najpierw cel i zakres pracy, następnie przejdę przez wymagania, wykorzystane technologie, architekturę systemu, bazę danych, najważniejsze mechanizmy oraz testy.

Slajd 2. Spis treści

Prezentacja została ułożona tak, aby najpierw pokazać, jaki problem rozwiązuje aplikacja i jakie założenia przyjęto podczas jej projektowania. Następnie przedstawione zostaną wymagania funkcjonalne i niefunkcjonalne oraz krótkie porównanie podobnych narzędzi.

W dalszej części omówię podstawy technologiczne frontendu i backendu, sposób działania autentykacji i autoryzacji, wybór bazy danych oraz architekturę aplikacji. Na końcu przejdę do najważniejszych mechanizmów, testów, krótkiej demonstracji aplikacji, podsumowania i bibliografii.

Slajd 3. Cel pracy

Celem pracy było zaprojektowanie i zaimplementowanie aplikacji internetowej wspierającej indywidualną organizację zadań. Aplikacja miała pozwalać użytkownikowi tworzyć taski, checklisty, notatki oraz projekty, a następnie łączyć te elementy ze sobą w wygodny sposób.

Istotnym założeniem było ograniczenie rozproszenia informacji. W praktyce użytkownik często korzysta z kilku osobnych narzędzi: osobno zapisuje zadania, osobno notatki, a jeszcze gdzie indziej przechowuje większe projekty. W mojej aplikacji te elementy zostały zebrane w jednym miejscu.

Drugim ważnym celem było przygotowanie systemu, który będzie prostszy niż rozbudowane narzędzia zespołowe. Aplikacja nie próbuje zastąpić dużych platform do zarządzania firmą, tylko skupia się na codziennej pracy pojedynczego użytkownika.

Slajd 4. Zakres pracy

Zakres pracy obejmował kilka etapów. Najpierw przeanalizowane zostały podobne aplikacje oraz potrzeby użytkownika, aby określić, jakie funkcje będą najważniejsze w pierwszej wersji systemu.

Następnie omówione zostały technologie wykorzystane w projekcie aplikacji. W pracy opisano zarówno warstwę frontendową, jak i backendową, a także sposób przechowywania danych.

Kolejnym elementem było przygotowanie projektu interfejsu użytkownika na podstawie scenariuszy przypadków użycia. Dzięki temu widoki aplikacji nie powstawały przypadkowo, tylko wynikały z czynności, które użytkownik faktycznie ma wykonywać.

Ostatnia część zakresu obejmowała implementację kodu po stronie frontendu i backendu oraz przygotowanie testów jednostkowych dla wybranych elementów aplikacji.

Slajd 5. Wymagania funkcjonalne

Najważniejszym wymaganiem funkcjonalnym było logowanie oraz tworzenie konta użytkownika przez zewnętrznego dostawcę. Dzięki temu aplikacja nie musi samodzielnie przechowywać haseł, a użytkownik może zalogować się przy użyciu istniejącego konta.

System umożliwia tworzenie, edycję, odczyt oraz archiwizację elementów aplikacji. Dotyczy to przede wszystkim tasków, checklist, notatek i projektów. Archiwizacja została wybrana zamiast trwałego usuwania, ponieważ pozwala zachować spójność danych i powiązań.

Ważną funkcją jest przypinanie wybranych elementów do strony głównej. Dzięki temu użytkownik może szybko wrócić do rzeczy, które są dla niego aktualnie najważniejsze. Aplikacja pozwala również łączyć ze sobą różne elementy, na przykład taski z projektami albo notatki z innymi zasobami.

Projekty mogą być wyświetlane w formie tablicy Kanban, co ułatwia śledzenie postępu prac. Dodatkowo użytkownik może wyszukiwać, filtrować oraz sortować dane, co ma znaczenie wtedy, gdy liczba elementów w aplikacji rośnie.

Slajd 6. Uzupełnienie wymagań funkcjonalnych

Ten slajd traktuję jako miejsce na krótkie dopowiedzenie, że wymagania funkcjonalne zostały dobrane pod konkretny sposób pracy użytkownika. Aplikacja nie ogranicza się do samej listy zadań, tylko pozwala budować kontekst wokół pracy: projekt może zawierać taski, checklista może zostać przypięta do dashboardu, a notatki mogą uzupełniać inne elementy.

Warto podkreślić, że funkcje wyszukiwania i filtrowania nie są dodatkiem kosmetycznym. Przy większej liczbie tasków albo checklist stają się mechanizmem potrzebnym do sprawnego korzystania z aplikacji.

Slajd 7. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne dotyczyły przede wszystkim jakości działania aplikacji. Interfejs miał być czytelny, responsywny i spójny graficznie, ponieważ aplikacja do organizacji pracy powinna pomagać użytkownikowi, a nie dokładać mu chaosu.

Drugim ważnym wymaganiem było bezpieczeństwo danych. Użytkownik powinien mieć dostęp wyłącznie do własnych zasobów, dlatego w operacjach backendowych sprawdzany jest identyfikator właściciela danych.

Kolejnym wymaganiem była walidacja danych przesyłanych do API. Dzięki temu backend nie zapisuje przypadkowych albo niepełnych danych, nawet jeśli żądanie zostanie wysłane poza interfejsem aplikacji.

Znaczenie miała też struktura kodu. Projekt został napisany tak, aby można było rozwijać go dalej, na przykład dodając kolejne typy elementów, rozszerzając wyszukiwanie albo rozbudowując mechanizmy uprawnień.

Slajd 8. Podobne aplikacje

Przed rozpoczęciem projektowania przeanalizowane zostały popularne aplikacje do organizacji pracy, takie jak Trello, Asana i ClickUp. Każda z nich rozwiązuje podobny problem, ale robi to w trochę inny sposób.

Trello jest najbardziej kojarzone z tablicami Kanban i prostym układem kart. Asana jest bardziej rozbudowana i mocniej nastawiona na zarządzanie projektami zespołowymi. ClickUp oferuje

bardzo wiele funkcji i widoków, ale właśnie przez to może być zbyt złożony dla użytkownika indywidualnego.

Własna aplikacja została zaprojektowana jako prostsza alternatywa. Jej celem nie było kopiowanie wszystkich funkcji tych narzędzi, lecz wybranie rozwiązań najbardziej przydatnych w indywidualnej organizacji zadań.

Slajd 9. Uzupełnienie podobnych aplikacji

Jeżeli na slajdzie znajduje się grafika lub porównanie narzędzi, warto omówić je przez wskazanie różnic między aplikacjami. Trello można przedstawić jako narzędzie najprostsze i najbardziej wizualne, Asanę jako narzędzie mocniejsze organizacyjnie, a ClickUp jako najbardziej rozbudowane.

Na tym tle moja aplikacja znajduje się bliżej prostoty Trello, ale jednocześnie rozszerza zakres pracy o checklisty, notatki, projekty i dashboard z przypiętymi elementami. Dzięki temu użytkownik może przechowywać więcej typów informacji bez wchodzenia w poziom złożoności typowy dla dużych systemów zespołowych.

Slajd 10. Podstawy technologiczne frontendu

Frontend aplikacji został oparty na technologiach webowych, które umożliwiają budowę interaktywnego interfejsu użytkownika. Podstawą są HTML, CSS i JavaScript, czyli technologie odpowiadające kolejno za strukturę, wygląd oraz zachowanie strony.

W projekcie ważną rolę pełni React. Biblioteka ta pozwala budować interfejs z komponentów, czyli mniejszych, wielokrotnie używanych części aplikacji. Dzięki temu formularze, karty obiektów czy sekcje dashboardu mogą być rozwijane w bardziej uporządkowany sposób.

Do projektu wykorzystano również TypeScript, który pomaga kontrolować typy danych przekazywanych między komponentami i funkcjami. Stylowanie interfejsu zostało zrealizowane przy użyciu Tailwind CSS, co pozwoliło szybko tworzyć spójne widoki za pomocą klas narzędziowych.

Slajd 11. Podstawy technologiczne backendu

Backend aplikacji został oparty na mechanizmach dostępnych w Next.js. Dzięki temu frontend i backend znajdują się w jednym projekcie, ale nadal pełnią odrębne role. Interfejs odpowiada za prezentację danych, a część serwerowa za obsługę żądań, walidację i komunikację z bazą danych.

W aplikacji wykorzystane zostały Route Handlers, czyli mechanizm Next.js pozwalający tworzyć endpointy API. Endpointy obsługują między innymi operacje pobierania, tworzenia, aktualizacji i archiwizacji danych.

Ważnym elementem backendu jest też walidacja danych wejściowych. Dzięki niej serwer sprawdza, czy przesłane dane mają poprawny format, zanim zostaną zapisane w bazie.

Slajd 12. Autentykacja i autoryzacja

W aplikacji rozróżniono dwa pojęcia: autentykację i autoryzację. Autentykacja odpowiada na pytanie, kim jest użytkownik, natomiast autoryzacja określa, do jakich danych i operacji ten użytkownik ma dostęp.

Logowanie zostało oparte na zewnętrznych dostawcach, takich jak Google i GitHub. Do obsługi tego mechanizmu wykorzystano Auth.js, znany wcześniej jako NextAuth.js. Dzięki temu nie trzeba było tworzyć całego przepływu logowania od podstaw.

Po zalogowaniu aplikacja korzysta z danych sesji użytkownika. Przy operacjach na danych backend sprawdza, czy dokument należy do aktualnego użytkownika, najczęściej przez pole `ownerId`. W ten sposób jeden użytkownik nie powinien mieć dostępu do zasobów innej osoby.

Slajd 13. Wybór bazy danych

Do przechowywania danych wybrano MongoDB, czyli dokumentową bazę danych. Decyzja ta wynikała z charakteru aplikacji, ponieważ taski, checklisty, notatki i projekty mają elastyczną strukturę oraz różne typy powiązań.

W relacyjnej bazie danych wiele takich zależności wymagałoby dodatkowych tabel pomocniczych. W MongoDB część danych można zapisać jako dokumenty, a część powiązać przez identyfikatory. Dobrze pasuje to na przykład do checklist, które składają się z wielu elementów,

oraz do projektów z własnymi kolumnami Kanban.

W projekcie wykorzystano również Mongoose, czyli bibliotekę, która pozwala opisywać schematy dokumentów i pracować z bazą w bardziej uporządkowany sposób. Dzięki temu mimo elastyczności MongoDB struktura danych pozostaje kontrolowana przez modele aplikacji.

Slajd 14. Ogólna architektura aplikacji

Na slajdzie z ogólną architekturą aplikacji należy zwrócić uwagę na podział systemu na warstwę klienta, warstwę serwerową oraz bazę danych. Użytkownik korzysta z aplikacji w przeglądarce, a interfejs komunikuje się z backendem przez endpointy API.

Backend odpowiada za sprawdzenie sesji użytkownika, walidację danych, wykonanie logiki biznesowej oraz zapis lub odczyt informacji z bazy. Baza danych nie jest dostępna bezpośrednio z poziomu przeglądarki, co zwiększa bezpieczeństwo i pozwala kontrolować operacje po stronie serwera.

W architekturze widoczna jest również rola zewnętrznych dostawców logowania. Są oni wykorzystywani tylko w procesie autentykacji, natomiast dane aplikacji są przechowywane w MongoDB.

Slajd 15. Diagram komponentów aplikacji

Diagram komponentów pokazuje, z jakich głównych części składa się system. Po stronie użytkownika znajduje się interfejs aplikacji, zbudowany z widoków i komponentów React. Ta część odpowiada za formularze, listy, dashboard, tablicę Kanban oraz interakcje użytkownika.

Po stronie backendu znajdują się endpointy API oraz funkcje pomocnicze odpowiedzialne za obsługę sesji, walidację i przygotowanie odpowiedzi. Endpointy komunikują się z modelami Mongoose, które reprezentują kolekcje w bazie danych.

Diagram dobrze pokazuje, że aplikacja nie jest jedną płaską strukturą. Każda warstwa ma własną odpowiedzialność, a dane przepływają między nimi w określonym kierunku.

Slajd 16. Diagramy przepływu danych

Diagram przepływu danych poziomu 0 pokazuje system w uproszczony sposób, jako jeden główny proces. Z jednej strony znajduje się użytkownik, który wykonuje akcje w aplikacji, a z drugiej baza danych i zewnętrzni dostawcy logowania.

Ten slajd można potraktować jako ogólne wyjaśnienie, skąd dane wchodzą do systemu i dokąd trafiają. Użytkownik wysyła żądania przez interfejs, aplikacja przetwarza je po stronie serwera, a następnie pobiera lub zapisuje dane w bazie.

Na tym poziomie nie analizujemy jeszcze szczegółowych endpointów ani modeli. Najważniejsze jest pokazanie granic systemu oraz jego relacji z otoczeniem.

Slajd 17. Diagram przepływu danych poziomu 1

Diagram poziomu 1 rozwija poprzedni widok i pokazuje system bardziej szczegółowo. Można na nim wyróżnić proces autentykacji, obsługę elementów aplikacji, komunikację z bazą danych oraz zwracanie danych do interfejsu użytkownika.

Ten diagram pomaga wyjaśnić, co dzieje się po wykonaniu typowej akcji w aplikacji. Na przykład po utworzeniu taska formularz wysyła dane do endpointu API, backend sprawdza użytkownika i poprawność danych, zapisuje dokument w bazie, a potem zwraca odpowiedź do frontendu.

Dzięki takiemu rozbiciu łatwiej pokazać, że aplikacja nie tylko wyświetla formularz, ale obsługuje cały przepływ: od akcji użytkownika, przez walidację, po zapis i aktualizację widoku.

Slajd 18. Schemat bazy danych

Schemat bazy danych przedstawia najważniejsze kolekcje wykorzystywane w aplikacji. Wśród nich znajdują się między innymi accounts, tasks, projects, checklists, notes oraz pins.

Warto podkreślić, że jest to schemat dokumentowy przygotowany pod MongoDB i Mongoose, a nie klasyczny relacyjny diagram ERD. Relacje między dokumentami są realizowane głównie przez identyfikatory, na przykład ownerId, projectId albo tablice identyfikatorów.

Kolekcja accounts wspiera mechanizm logowania przez zewnętrznych dostawców. Kolekcja projects przechowuje dane projektów oraz ustawienia kolumn Kanban. Kolekcja pins odpowiada za elementy przypięte do dashboardu, czyli za szybki dostęp do najważniejszych zasobów.

Slajd 19. Najważniejsze mechanizmy aplikacji

Jednym z najważniejszych mechanizmów aplikacji jest przypinanie elementów do dashboardu. Pozwala ono użytkownikowi wyróżnić te taski, checklisty, notatki albo projekty, do których chce wracać najczęściej.

Drugim istotnym mechanizmem jest powiązywanie elementów ze sobą. Task może należeć do projektu, notatka może uzupełniać inne elementy, a projekt może zawierać zadania prezentowane w różnych widokach.

W aplikacji ważną rolę pełni również tablica Kanban. Zadania są przypisywane do kolumn statusów, co ułatwia śledzenie postępu pracy. Oprócz tego zastosowano walidację danych, archiwizację elementów oraz zapisywanie wybranych zdarzeń aktywności.

Slajd 20. Testy

W projekcie przygotowano testy jednostkowe dla wybranych mechanizmów związanych z taskami. Testy uruchamiane są poleceniem `npm test`, które wykonuje zestaw testów przy użyciu narzędzia Vitest.

Testy obejmują między innymi walidację danych taska. Dzięki temu można sprawdzić, czy aplikacja przyjmuje poprawne dane i odrzuca przypadki niezgodne z założonym schematem.

Przetestowane zostały również wybrane endpointy API odpowiedzialne za tworzenie i aktualizację tasków. W testach zastosowano atrapy zależności, czyli mocki, dzięki czemu można było sprawdzić logikę handlerów bez łączenia się z rzeczywistą bazą danych.

Slajd 21. Demonstracja aplikacji

W tym miejscu prezentacji przechodzę do krótkiego pokazu działania aplikacji. Najpierw warto pokazać logowanie, ponieważ od niego zaczyna się dostęp do danych użytkownika.

Następnie można omówić dashboard i przypięte elementy. To dobry moment, aby pokazać, że aplikacja nie jest zwykłą listą zadań, tylko pozwala wyróżniać najważniejsze zasoby.

W dalszej części demonstracji warto utworzyć task, pokazać checklistę oraz przejść do projektu. W projekcie najlepiej pokazać oba sposoby prezentacji zadań: listę oraz tablicę Kanban.

Slajd 22. Podsumowanie

W ramach pracy udało się osiągnąć główny cel, czyli przygotować działającą aplikację internetową do organizacji zadań. System pozwala użytkownikowi tworzyć i porządkować taski, checklisty, notatki oraz projekty.

Projekt pokazał również praktyczne zastosowanie technologii omawianych w pracy, takich jak React, Next.js, TypeScript, MongoDB, Mongoose i Auth.js. Ważne było nie tylko stworzenie interfejsu, ale także zaprojektowanie przepływu danych, autoryzacji, struktury bazy i testów.

Aplikację można dalej rozwijać. Najbardziej naturalne kierunki to rozszerzenie testów o scenariusze integracyjne, dodanie mechanizmów automatyzujących, powiadomień, współdzielenia projektów oraz dokładniejszego modelu uprawnień.

Slajd 23. Bibliografia

Na slajdzie bibliograficznym zostały wskazane najważniejsze źródła wykorzystane przy omawianiu technologii i architektury. Wśród nich znajduje się dokumentacja Next.js, ponieważ framework ten jest podstawą zarówno routingu, jak i części serwerowej aplikacji.

Drugim ważnym źródłem jest dokumentacja MongoDB, ponieważ projekt korzysta z dokumentowego modelu danych. W bibliografii znalazło się również źródło dotyczące architektury modularnej, które pomagało opisać sposób uporządkowania aplikacji.

W trakcie wypowiedzi nie trzeba szczegółowo omawiać każdej pozycji bibliograficznej. Wystarczy zaznaczyć, że źródła zostały dobrane do konkretnych fragmentów pracy: technologii, architektury oraz bazy danych.

Slajd 24. Dziękuję za uwagę

Na koniec dziękuję komisji za uwagę i sygnalizuję gotowość do pytań. W tym miejscu warto przejść do rozmowy spokojnie, bez dopowiadania kolejnych technicznych szczegółów, chyba że komisja o nie zapyta.

Jeżeli pojawi się pytanie o projekt, najlepiej odpowiadać przez konkretne przykłady z aplikacji: jak działa logowanie, jak backend sprawdza właściciela danych, jak wygląda zapis taska albo dlaczego wybrano MongoDB zamiast bazy relacyjnej.